Digital Images & the Pipeline

Week 1 — What an image really is

Practical Image Processing

Lecture 1 ▪ 60–70 min

Where this course is going

By the end you will build DocScan: a photo of a document $\rightarrow$ a clean, deskewed, machine-readable scan, then read it with a small neural net.

- Tools: **Python + OpenCV + NumPy**.
- Weeks 1–11: classical image processing.
- Weeks 12–14: an introduction to deep learning for vision.

The habit we build

Understand the pixels and the math *first*; call the library second; reach for a neural net only when classical methods run out.

The image-processing pipeline



Key idea

Every task in this course is some path through this pipeline. DocScan walks all of it.

Roadmap

What is a digital image?

Color

Images in code

In-class exercise

Formats & storage

Deep dive: the imaging model

Wrap-up

What is a digital image?

An image is a function, sampled onto a grid

- A continuous image is a function $f(x, y)$ = brightness at position (x, y).
- A **digital** image samples it on a grid and rounds each value: **pixels**.
- A grayscale image is a 2D array; a color image is 3 such arrays (channels).

Math

$f(x, y)$, with $x = 0 \dots W-1$, $y = 0 \dots H-1$, and each value in $\{0, \dots, 255\}$ (8-bit).

Pixels are just numbers

A tiny 5×5 patch of intensities:

10	20	120	200	240
30	60	130	180	230
50	90	150	190	220
70	110	160	200	250
90	130	170	210	255

- 0 = black, 255 = white, values between = gray.
- "Processing" an image = doing arithmetic on these numbers.
- There is no magic: every operation is math on an array.

A real image from our dataset



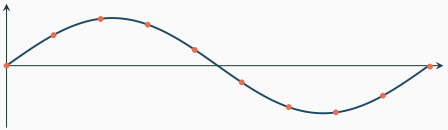
original (color) — a water-meter numerator



grayscale — one channel, same structure

This 48×196 crop is our running example. Reading these digits is exactly what the DocScan capstone builds toward. (Data: `E:/data/abfa_image/numerators`.)

Sampling: how finely we measure space



Sampling = choosing grid points (resolution). Too few samples and detail is lost — or worse, faked (aliasing, deep dive). More pixels = more spatial detail, more data.

Quantization: how finely we measure brightness

- **Quantization** = rounding each sample to a finite set of levels (the bit depth).
- 8-bit = 256 levels (0–255); 1-bit = black/white; 16-bit = 65,536 (medical/RAW).
- Too few levels → visible **banding** in smooth gradients.

Note

Resolution (sampling) and bit depth (quantization) are the two axes of "how much image."

Resolution, size, and memory

- A $W \times H$ grayscale 8-bit image is $W \cdot H$ bytes; color is $3\times$ that.
- A 12-megapixel color photo $\approx$ 36 MB *uncompressed*.
- That's why formats compress — and why processing big images needs care (Week 11).

Key idea

Know your image's shape and dtype before anything else: `img.shape`, `img.dtype`.

Color

Color: three numbers per pixel

- **RGB**: red, green, blue channels; how screens emit light.
- A color pixel is a triple (R, G, B), each 0–255.
- Grayscale collapses color to one intensity — often the first step (less data, same structure).

Math

Luminance: $Y = 0.299R + 0.587G + 0.114B$ — green dominates because our eyes do.

Why convert to grayscale

- Many operations (edges, thresholds) need intensity, not color — 1 channel, 3× less work.
- DocScan works mostly in grayscale; color returns only when it carries information.
- But grayscale *discards* information — don't do it if hue matters (Week 8 color segmentation).

✕ Pitfall

Averaging $(R+G+B)/3$ is a common shortcut — but the weighted luminance is perceptually correct.

Other color spaces, and why

Space	Channels	Good for
RGB	red, green, blue	storage, display
Grayscale	intensity	most classical processing
HSV	hue, saturation, value	color-based selection (robust to lighting)
Lab	lightness, a, b	perceptual distance, color correction

Key idea

Choosing the right color space can make a hard problem easy — e.g. "find the red object" is trivial in HSV, painful in RGB.

HSV intuition

- **Hue**: which color (angle on the color wheel). **Saturation**: how vivid. **Value**: how bright.
- Separating color from brightness makes selection robust to shadows and lighting.
- "All the red pixels" = a hue range — one threshold, done.

Images in code

An image is a NumPy array

```python
import cv2, numpy as np
img = cv2.imread('page.jpg')          # BGR, shape (H, W, 3), dtype uint8
print(img.shape, img.dtype)           # e.g. (1200, 900, 3) uint8
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # (H, W)
print(gray[0, 0])                     # a single pixel's intensity, 0..255
```

✗ OpenCV gotcha

OpenCV loads images as **BGR**, not RGB. Forgetting this flips red and blue everywhere.

Pixels, slices, and vectorized ops

```
h, w = gray.shape
roi = gray[100:200, 50:150]           # a rectangular region (a view)
gray[gray < 50] = 0                    # vectorized: threshold dark pixels to black
brighter = np.clip(gray.astype(int) + 40, 0, 255).astype(np.uint8)
```

✔ Good practice

Never loop over pixels in Python — it's 100× slower. Use NumPy vectorization / OpenCV.

Show and save

```python
import matplotlib.pyplot as plt
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)); plt.axis('off'); plt.show()
cv2.imwrite('gray.png', gray)          # PNG = lossless; JPG would re-compress
```

Note

Convert BGR→RGB before `plt.imshow`, or your colors look wrong.

The uint8 overflow trap — seen on real data



+60 done right (clip): uniformly brighter



+60 in uint8: bright pixels *wrapped* to dark

✖ Pitfall

$200+100 = 44$, not 255. Cast to `int`, compute, `clip` to 0–255 — or use `cv2.add` (saturating). A "brighten" that darkens bright areas is this bug.

In-class exercise

In-class exercise (10 min)

Predict, then run

```
gray = np.array([[100, 150],[200, 250]], dtype=np.uint8)
a = gray + 60          # (1) what are the four values?
b = cv2.add(gray, 60)  # (2) and here?
c = gray.mean()        # (3) what does this represent?
```

1. Compute `a` by hand — watch the overflow.
2. How does `b` differ?
2. What is `c`, and what would a very low `c` tell you about the image?

Debrief

- `a` = `[[160,210],[4,54]]` — 200+60 and 250+60 *wrapped* (overflow).
- `b` = `[[160,210],[255,255]]` — `cv2.add` saturates (clamps to 255).
- `c` = mean intensity (≈175); a very low mean means a mostly dark image (under-exposed).

Key idea

The mean/histogram of an image tells you about exposure before you look at it — Week 2.

Formats & storage

Image formats: lossless vs lossy

Format	Compression	Use
PNG	lossless	masks, line art, intermediate results
JPEG	lossy	photos; small files, but artifacts
TIFF	lossless/none	archival, scientific, multi-page
BMP/RAW	none	simplicity / sensor data

Key idea

Never save intermediate results as JPEG — its artifacts corrupt later processing. Use PNG until the very end.

JPEG artifacts matter for processing

- JPEG discards high-frequency detail in 8×8 blocks — invisible to you, poison to edge detection.
- Re-saving a JPEG repeatedly compounds the damage (generation loss).
- For DocScan, work from the least-compressed source you can get.

Bit depth & dynamic range

- 8-bit is fine for display; 12–16-bit sensors keep detail in shadows/highlights.
- **Dynamic range** = ratio of brightest to darkest a sensor can capture.
- HDR combines exposures to exceed a single sensor's range (beyond our scope, good to know).

Deep dive: the imaging model

Sampling theory & aliasing

- **Nyquist**: to capture detail at frequency f, sample at $> 2f$. Under-sample and high frequencies masquerade as low ones.
- That's **aliasing**: moiré patterns, jagged edges, wagon-wheels spinning backwards.
- Fix: blur (low-pass) *before* down-sampling — remove detail you can't represent (Week 3, 6).

✗ Pitfall

Resizing an image down without blurring first creates aliasing artifacts that no later step can undo.

Gamma: pixels aren't linear light

- Displays and files are **gamma-encoded**: pixel value $\neq$ linear light intensity.
- Storing perceptually (more codes for dark tones, where eyes are sensitive) saves bits.
- For *correct* math (averaging, blending), sometimes linearize first — a common subtle bug.

Math

Encoded $\approx$ (linear)$^{1/\gamma}$, with $\gamma \approx 2.2$. We'll use gamma for contrast in Week 2.

Where pixels come from



Real sensors capture one color per pixel (a **Bayer** mosaic) and interpolate the rest. Noise, blur, and color casts enter here — which is exactly what we clean up (Weeks 4, 2).

Noise is physical, not a bug

- Photon counting is random → shot noise; electronics add read noise.
- Low light / high ISO = more noise. It's the raw material Week 4 removes.
- Understanding *where* noise comes from tells you *how* to model and remove it.

Key idea

Every artifact you'll fight — noise, blur, aliasing, color cast — has a physical origin in this pipeline.

Coordinates & conventions

- Images are indexed `[row, col]` = `[y, x]` — the *opposite* of math (x, y).
- Origin is top-left; y increases *downward*. This trips up everyone once.
- OpenCV points are `(x, y)`; NumPy indices are `[y, x]`. Keep them straight.

✕ Pitfall

Swapping x and y is the #1 beginner bug in image code. When something's transposed, check this first.

Wrap-up

What you should now be able to do

- Explain sampling, quantization, resolution, and bit depth.
- Load an image, inspect its shape/dtype, convert color, slice regions — vectorized.
- Choose a color space and format for a task; avoid the uint8/BGR/xy traps.
- Name where noise, blur, and aliasing physically come from.

This week's deliverable

Due before Week 2 (see `project/`)

1. Load 3 images; print shape, dtype, min/max/mean; convert to grayscale and to HSV.
2. Write a correct `brighten(img, k)` (no overflow) and compare to `cv2.add`.
3. Take a phone photo of a document — your DocScan test image. Report its properties.
4. Short note: for your photo, which stages of the pipeline will be hardest, and why?

An image is just numbers on a grid.

Everything else in this course is arithmetic on those numbers.